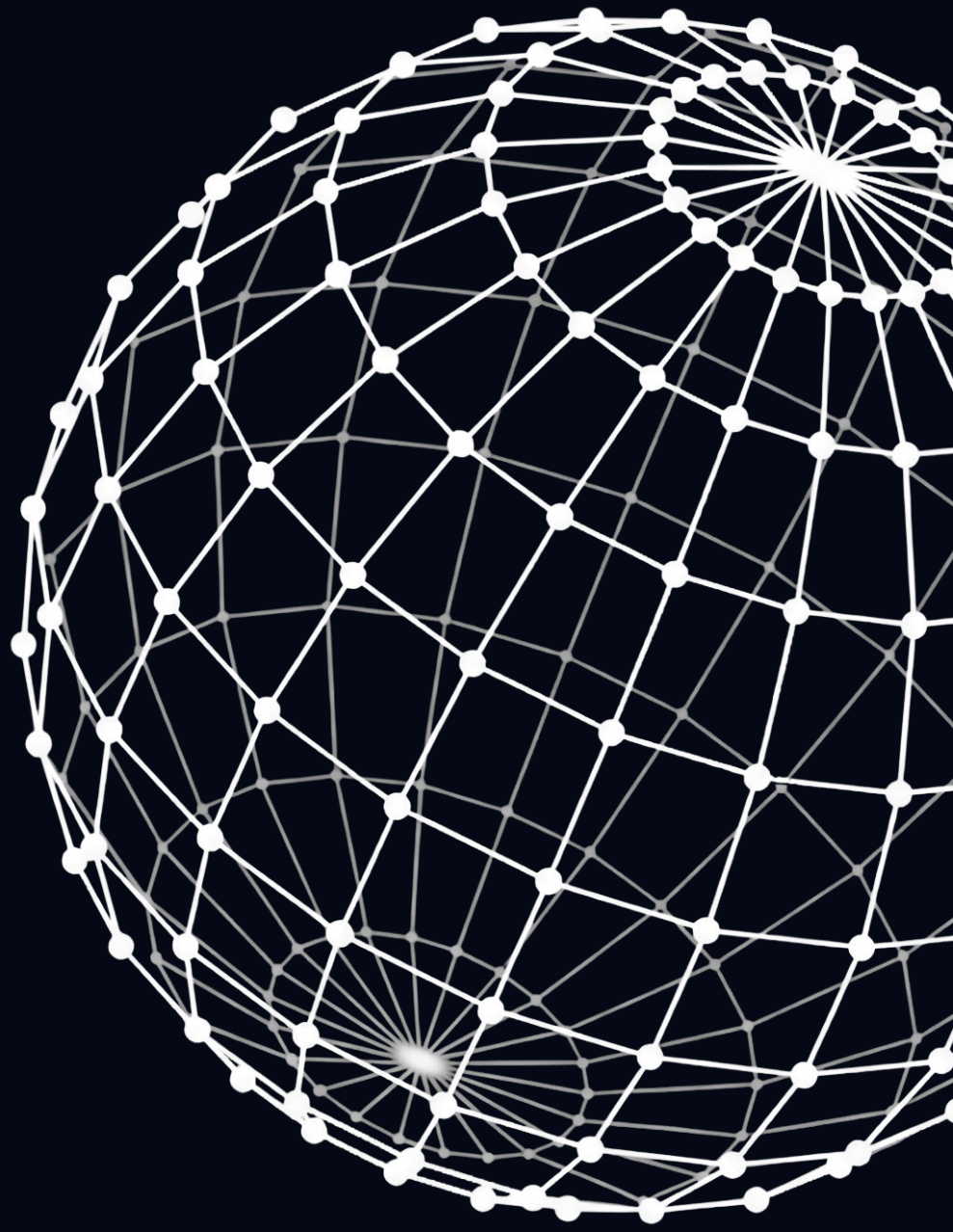
Git and GitHub

Version control for collaborative code

September 26, 2026



A project is a collection of files

A small project might contain:

```
app.py      helpers.py      tests.py      README.md
```

The files contain instructions and other text. Tools interpret or compile those instructions so a computer can run the program.

Changing the program usually means editing these files.

Three ways to share a project

Send the project files by email
Everyone downloads a copy and makes changes.

Work in one shared folder
The team edits the files in the same location.

Paste code into a chat message
People copy the pieces they need from a conversation.

Discuss: What could go wrong with each approach?

Where did our changes go?

Email copies	Someone edits an older copy. Two versions both seem current.
One shared folder	One person can overwrite another person’s work.
Chat snippets	The conversation is not a reliable history of the project.

Which version would you trust?



Who here has used version control?

Think about Google Docs

See earlier versions

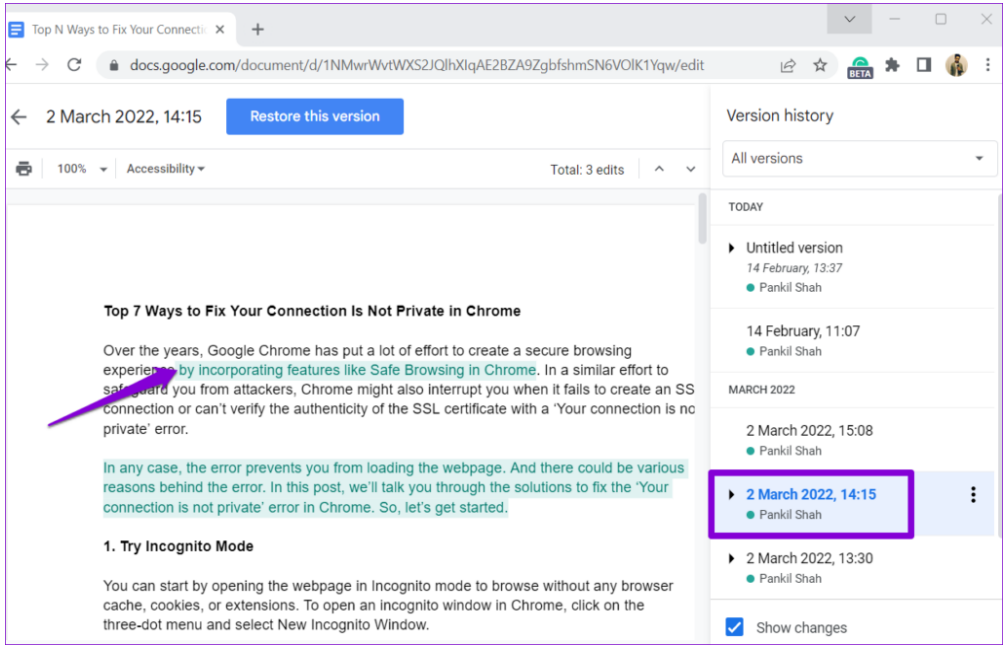
-Look back at a document before an edit.

See who changed it

-Understand how the document got to its current state

Restore previous work

-Recover text if an edit goes wrong



Git brings this idea to projects made of code files.

A history of the project

Keep a history

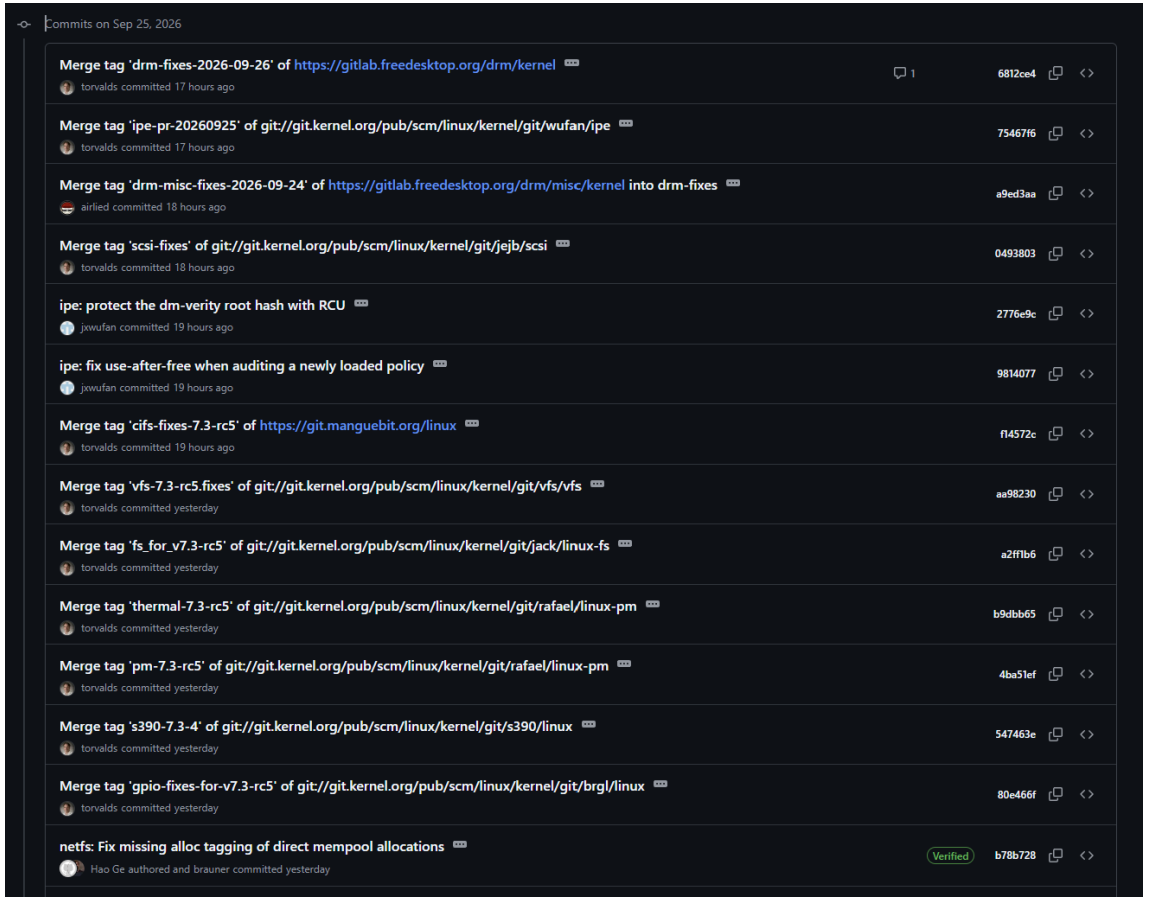
-See what changed, when it changed, and why.

Recover earlier work

-Return to a working version when something breaks.

Work in parallel

-Make separate changes, then combine them.



Git records changes to your files

You work in a folder called a repository. Git remembers snapshots of that folder called commits.

You can inspect history, compare changes, and go back to an earlier commit.

Git can work on your computer even without GitHub.

Commits are saved snapshots

One change should be one clear message

Edit the files

Make your change in the project.

Choose what to save

Git calls this staging. You select the changes for the snapshot.

Commit

Save the snapshot with a short description.

Example commit message: “Add login form”

A branch is a line of work

`main`	The shared project version
`feature/login`	A place to build the login feature while main stays as it is

A teammate can work on another branch at the same time. When the work is ready, the team can merge it into main.

Complete Example



GitHub hosts repositories online

Git

Tracks files and commits on your computer.

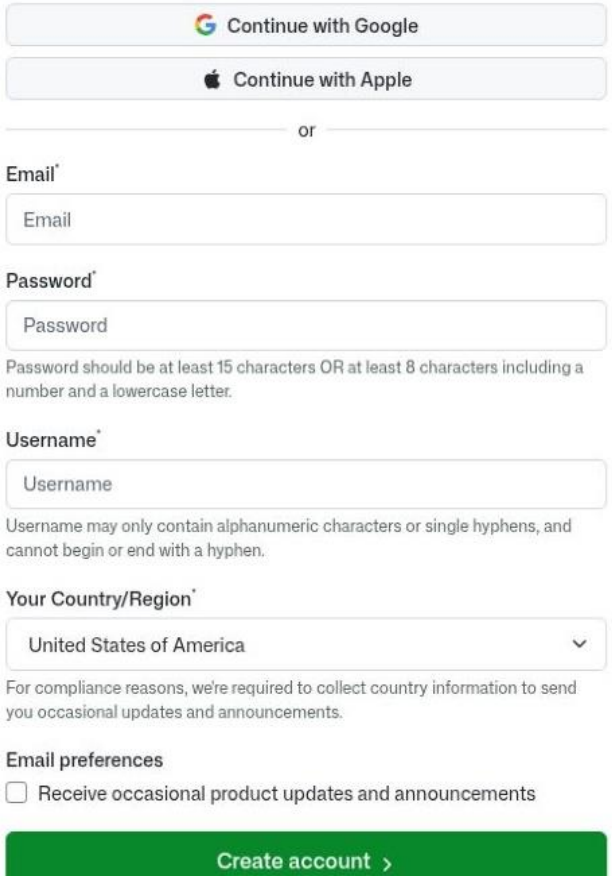
GitHub

Lets teammates share code, discuss changes, and review pull requests.

You use Git for the history. You use GitHub to collaborate around that history.

Create a GitHub account

Sign up at github.com



1. Enter your email and password.
2. Choose a username.
3. Create the account and verify your email.

The form may look slightly different when you sign up.

Six steps in the team workflow

01 **Get the project**

Open the repository on GitHub and get a copy.

02 **Make a branch**

Create a separate place for your change.

03 **Edit and commit**

Change the files and save a snapshot.

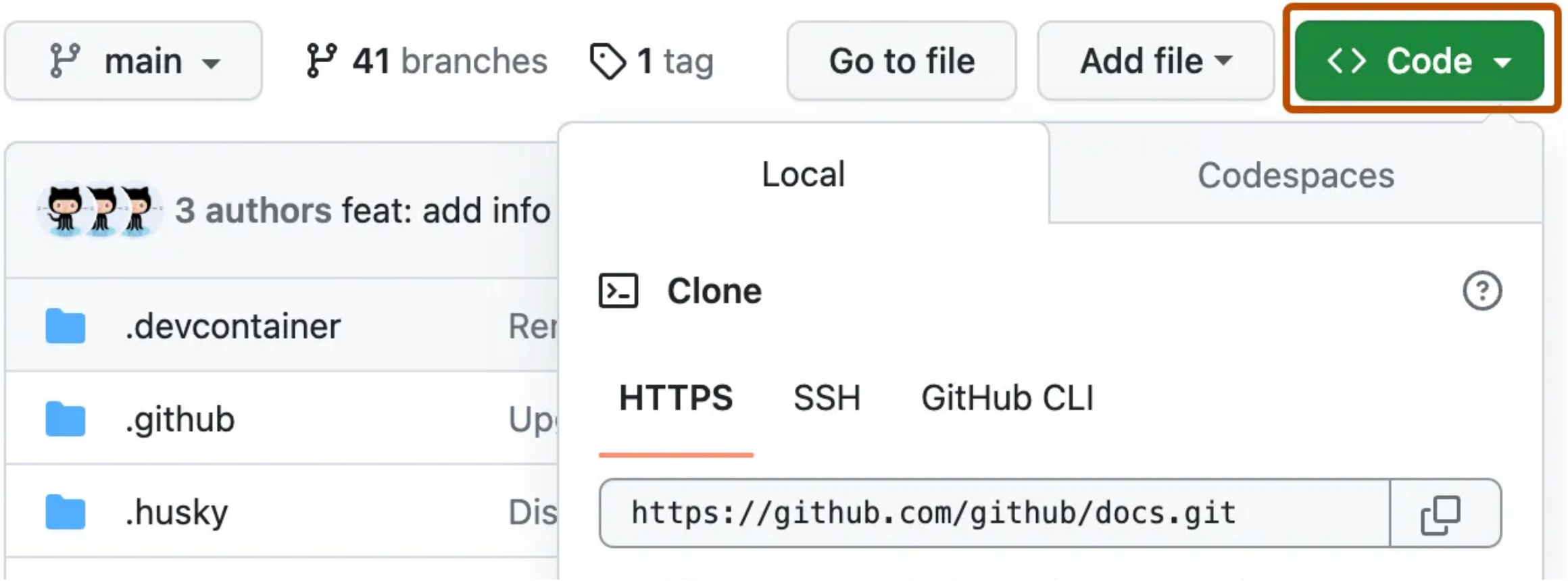
04 **Push the branch**

Share your branch with the online repository.

05 **Open a pull request**

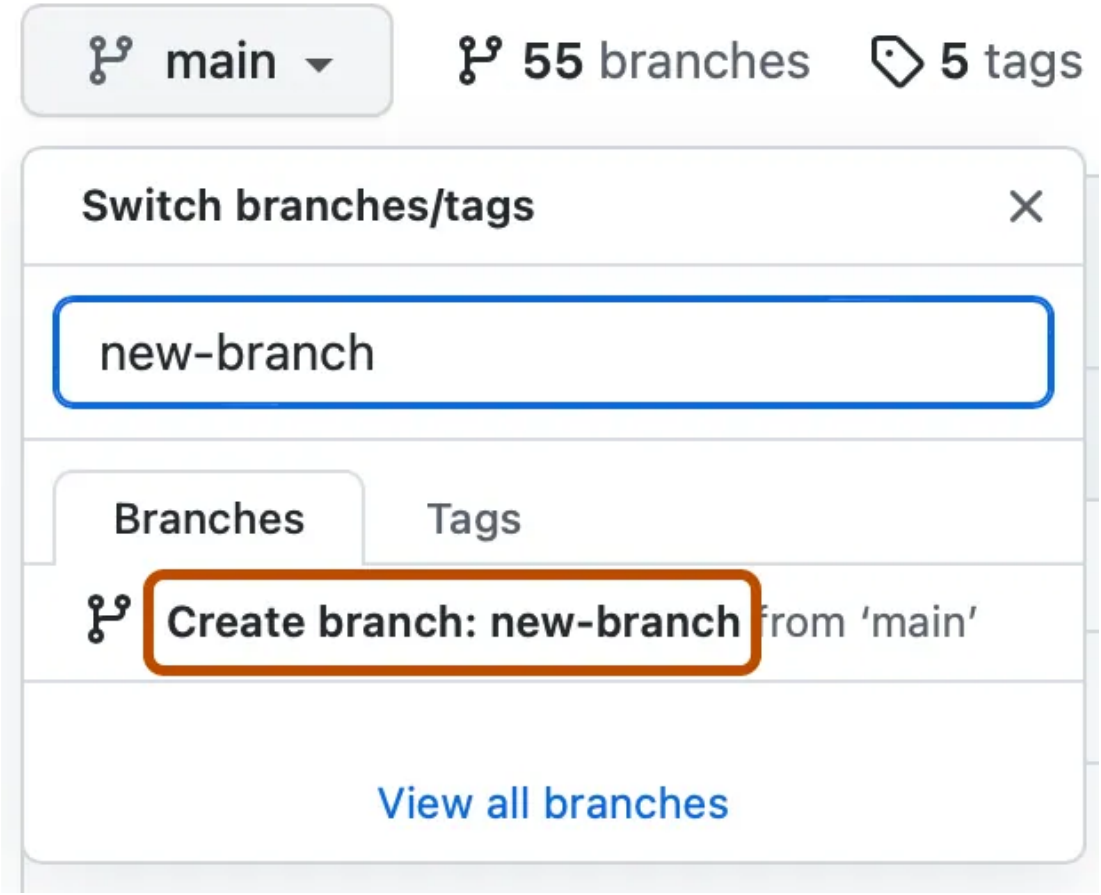
06 **Review and merge**

Get the project



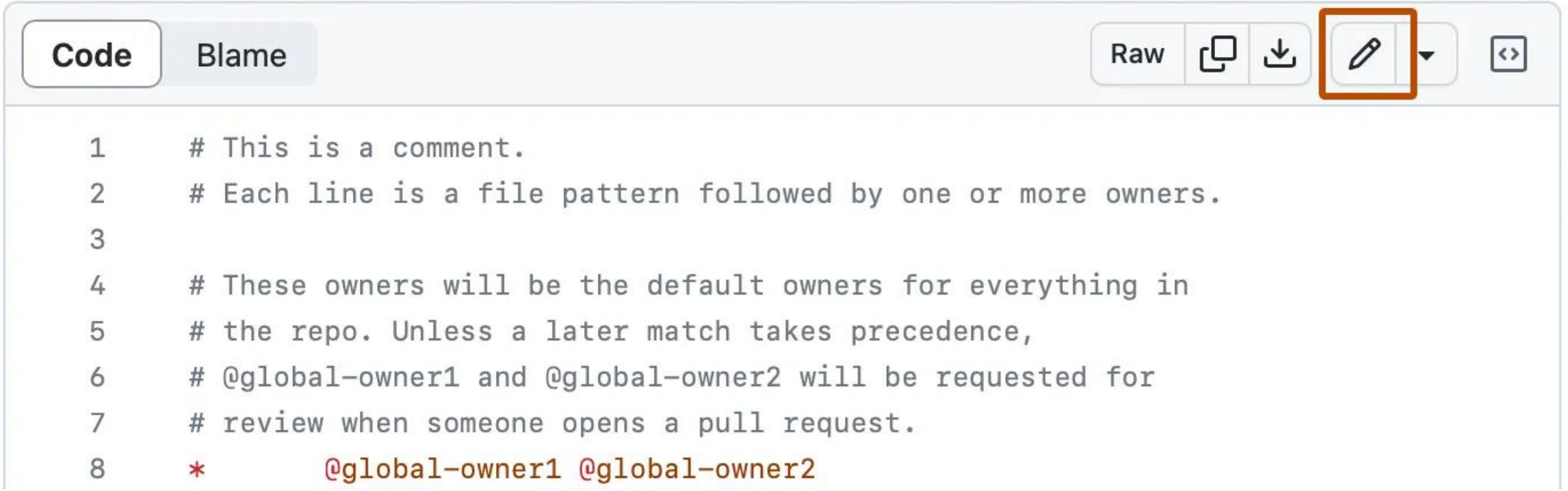
On the repository page, choose Code. Copy its URL or open it in GitHub Desktop.

Make a branch



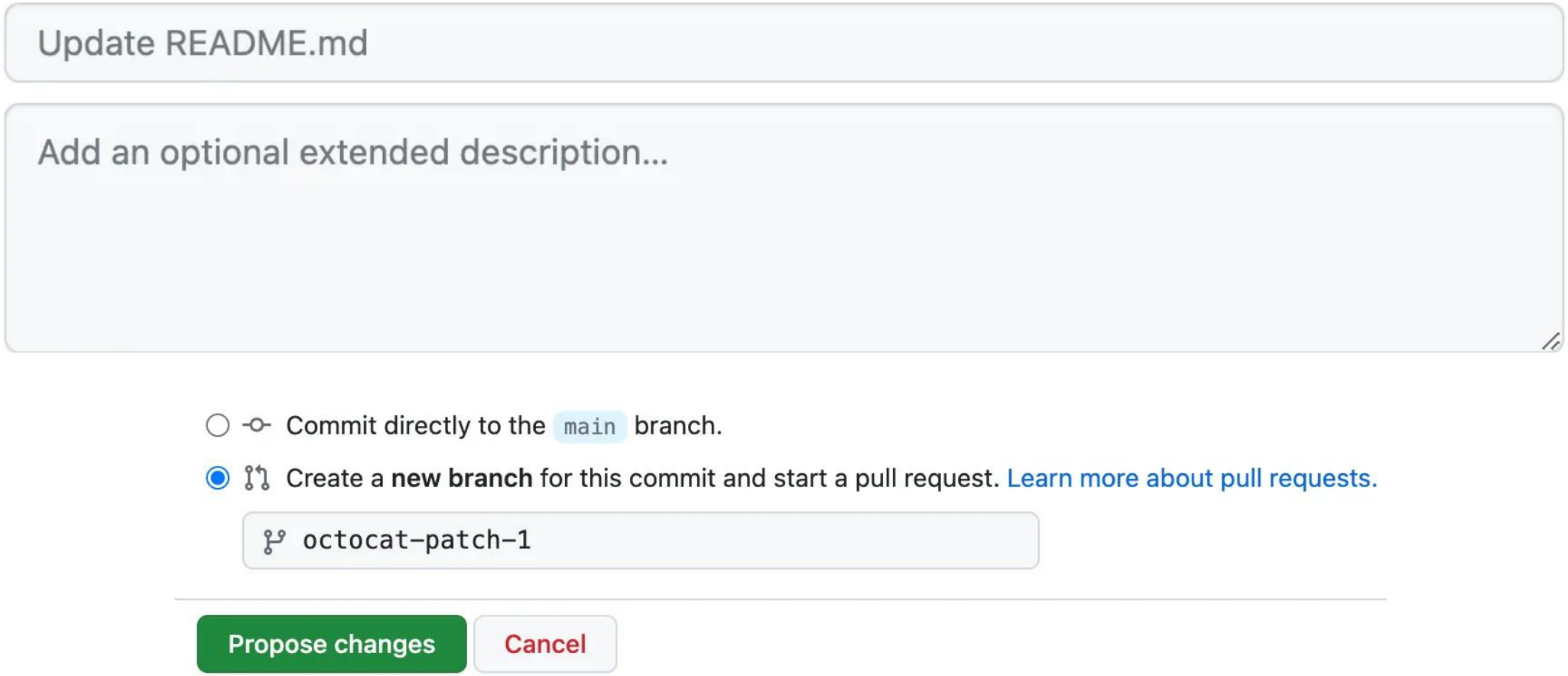
Use the branch menu, type a name, and create it from main.

Edit a file



Open a file and select the pencil icon. Make a small change to its text.

Commit the change



Write a message, choose the branch option, then select Propose changes. GitHub saves a commit.

What is a pull request?

“Can we add this change to main?”

Propose

Open a pull request from your branch into main.

Review

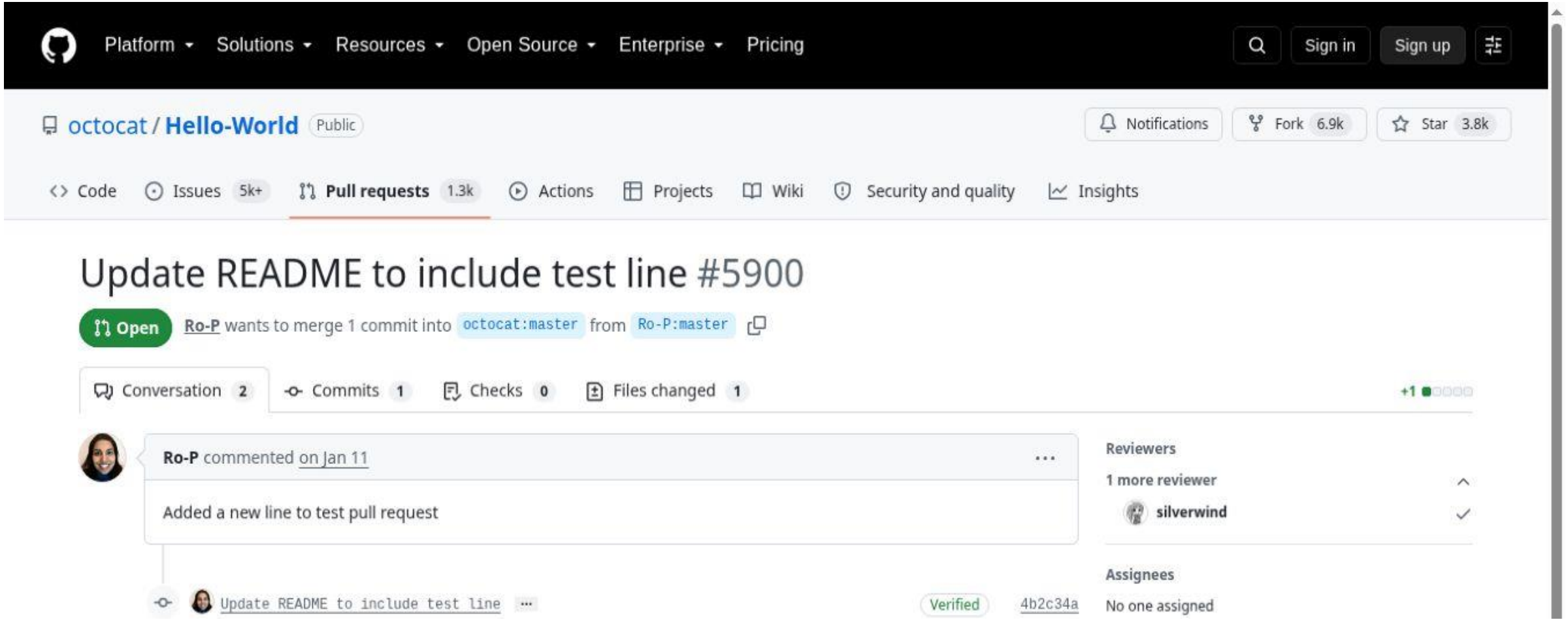
Teammates inspect the changed lines and leave comments.

Merge

When the change is ready, combine it with main.

PULL REQUEST EXAMPLE

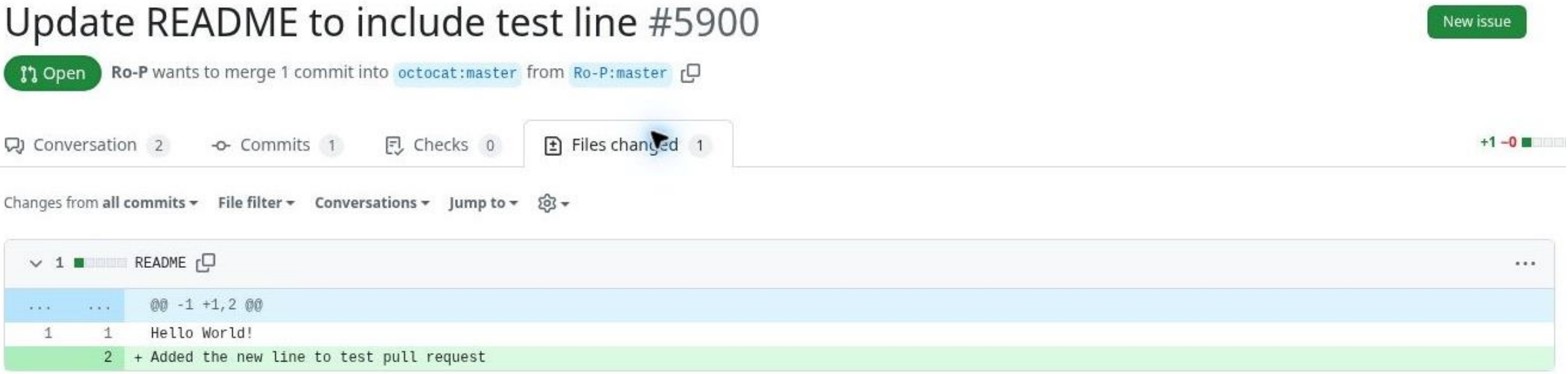
A real pull request



Title and branches show what will merge. The tabs hold discussion, commits, checks, and changed files.

PULL REQUEST EXAMPLE

What changed in the files?



The Files changed tab marks additions in green. Reviewers can inspect the exact lines before merging.

Example: octocat/Hello-World, pull request #5900



The End